MAE 204 Final Project

Control of a KukaBot

Instructor: Professor Tania Morimoto

TA: Jacobo Cervera-Torralba

Lab TA: Aedan Mangan

Authors:

Kaustubh Kanagalekar: kkanagalekar@ucsd.edu

Alexander Haken: ahaken@ucsd.edu

Report Summary:

This project was completed using a Jupyter Notebook in Python with the assistance of the Modern Robotics, Numpy, and Matplotlib libraries. There were three major functions that were designed to complete the project: TrajectoryGenerator, NextState, and FeedBackControl. The first function that was built was TrajectoryGenerator which took in inputs of various transformation matrices relating to the end effector configurations at each trajectory and outputted the corresponding cartesian trajectories. The second function that was built was the NextState function that took inputs of the current state, velocities, and magnitudes, outputting the next state of the robot based on odometry and elementary Euler steps. The third function that was built was FeedBackControl that took inputs of end effector current and next locations, proportional and integral matrices, and states and angles of the current state, outputting speeds for wheels and the arm, commanded twist, and error twist for plotting purposes. Finally, a wrapper script consisting of feedforward only, best feedback case, overshoot feedback case, and new task feedback case were tested, with their corresponding error twists plotted for analysis purposes.

FeedBackControl was designed a bit differently than what the instructions suggested: instead of an input consisting of the current, actual end effector configuration, it was calculated within the function using states and joint angles, the body axis twist, and forward kinematics. This was later used for finding the error twist between the desired configuration and the actual configuration. Additionally, the Jacobian was calculated inside the function, instead of outside the function. These differences do not undermine the final result, as the robot is still able to perform the necessary tasks flawlessly; it was more intuitive to perform the calculations internally to ensure the feedback portion of the project worked flawlessly.

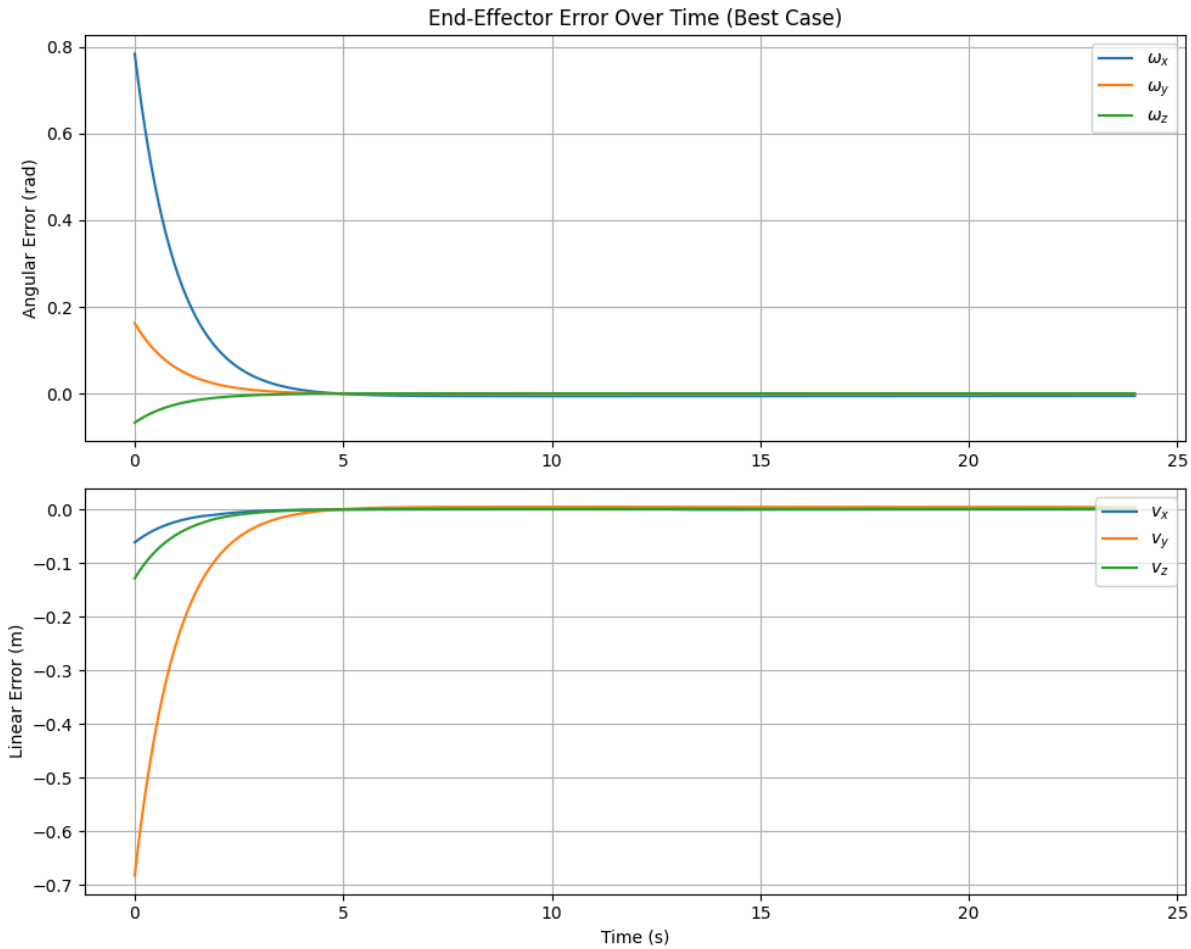
The lack of joint limit implementation within the main part of the project was a surprise as they play a significant part in a robot's performance in real life situations. For certain new case situations where the block was placed at a different location, these joint limits were vital to move the robot without any issues, however it was not implemented in this submission due to additional complexities rising up as a result of attempting to implement that functionality. Velocity limits were implemented, with a higher angular velocity ($100\ rad/s$) being permitted for the wheels compared to the arm ($25\ rad/s$) because it reflected the more realistic gear ratios for the robot. Earlier in the project, the same angular velocity limit was applied to all joints, but this led to over-actuation of the arm and under-actuation of the chassis since the end effector could travel much farther for an angular velocity on the arm compared to an angular velocity on the chassis. A simple Jacobian singularity avoidance was implemented by introducing a threshold for which the Jacobian would perform calculations. Any values less than the threshold would be set to zero, which would assist in the computation efficiency of the Jacobian.

Results:

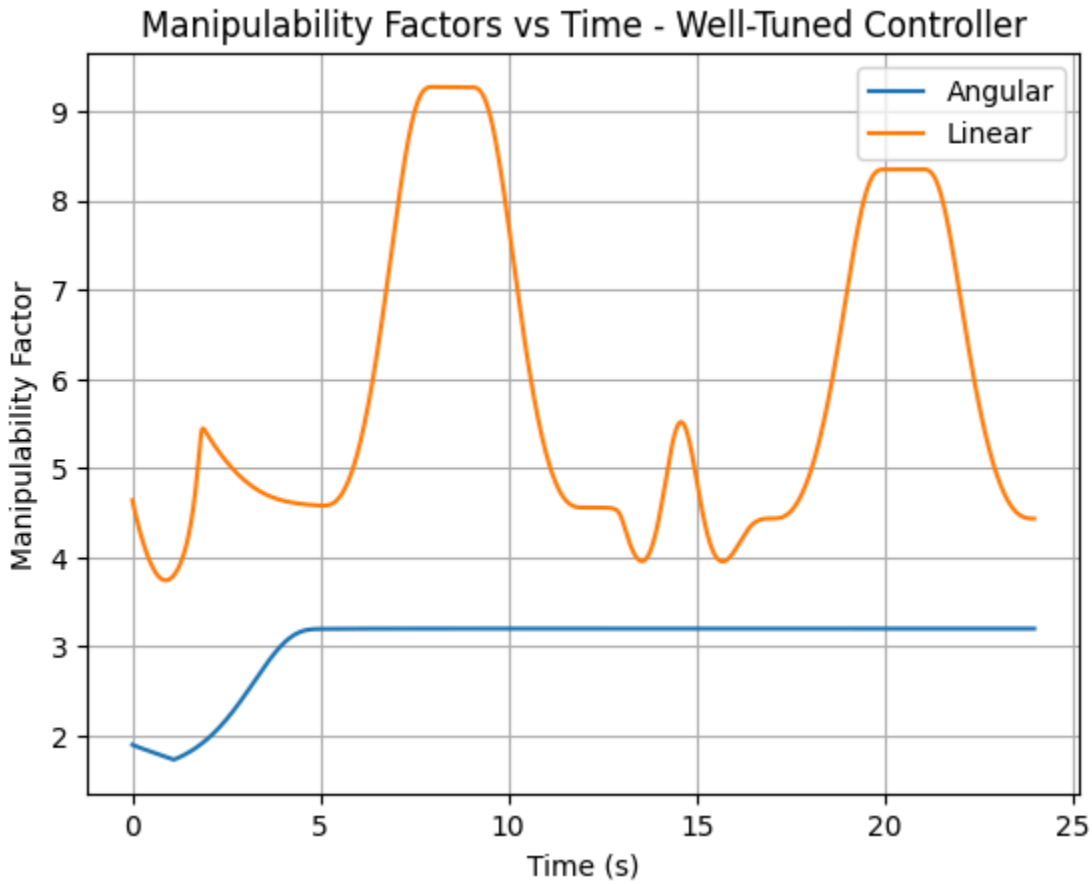
<u>Best Case</u>

For the best case scenario, a feedforward + PI controller was used with $K_p = 1.0$ and $K_i = 0.008$. The initial state of the robot was 45° off the original orientation, $-0.5\,m$ in the x direction, and $+0.3\,m$ in the y direction.

The error plots are shown below-



Manipulability Factors Plot:



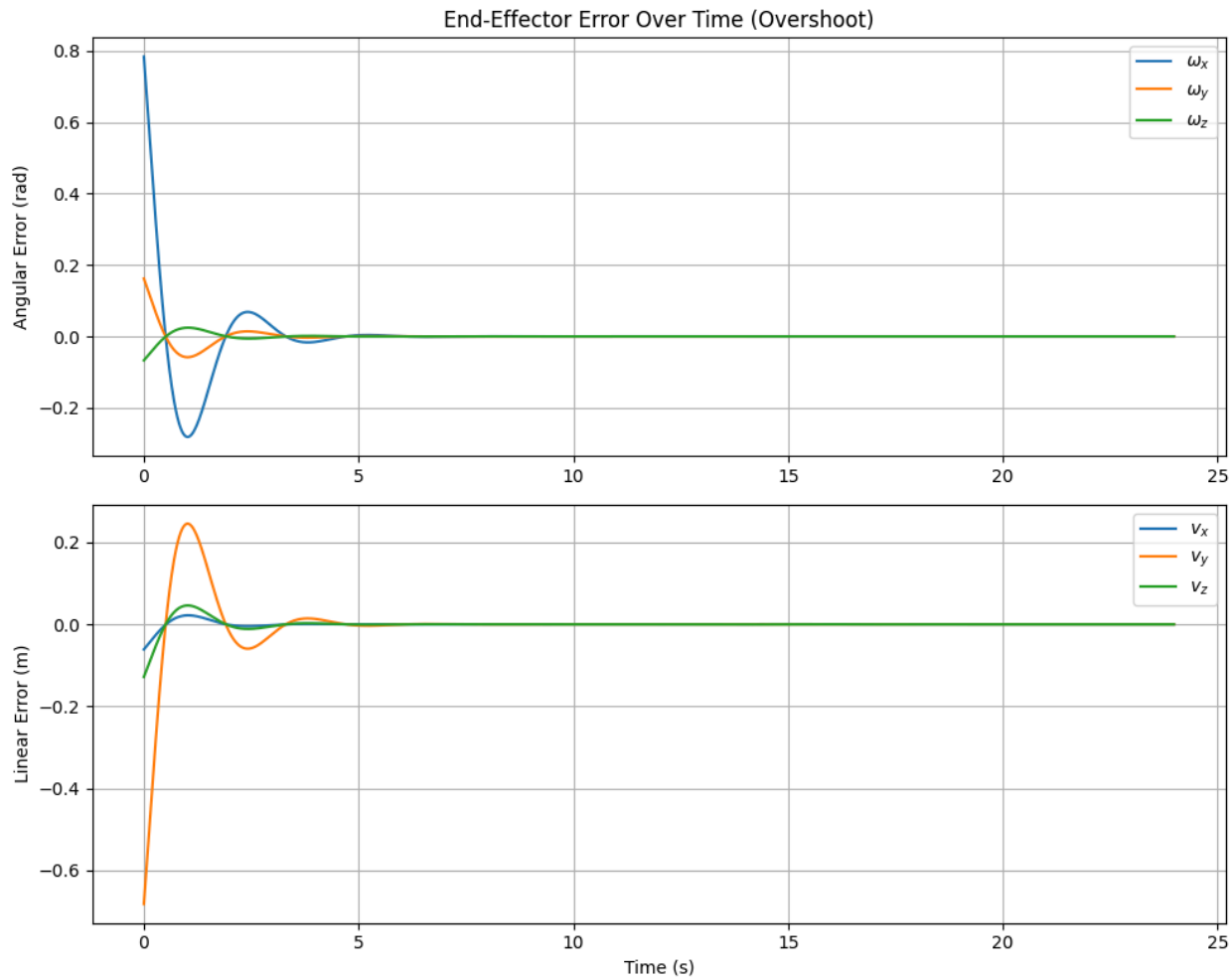
Youtube Video of Best Case:
https://youtu.be/lg1AVz0kbiA

Overshoot Case:

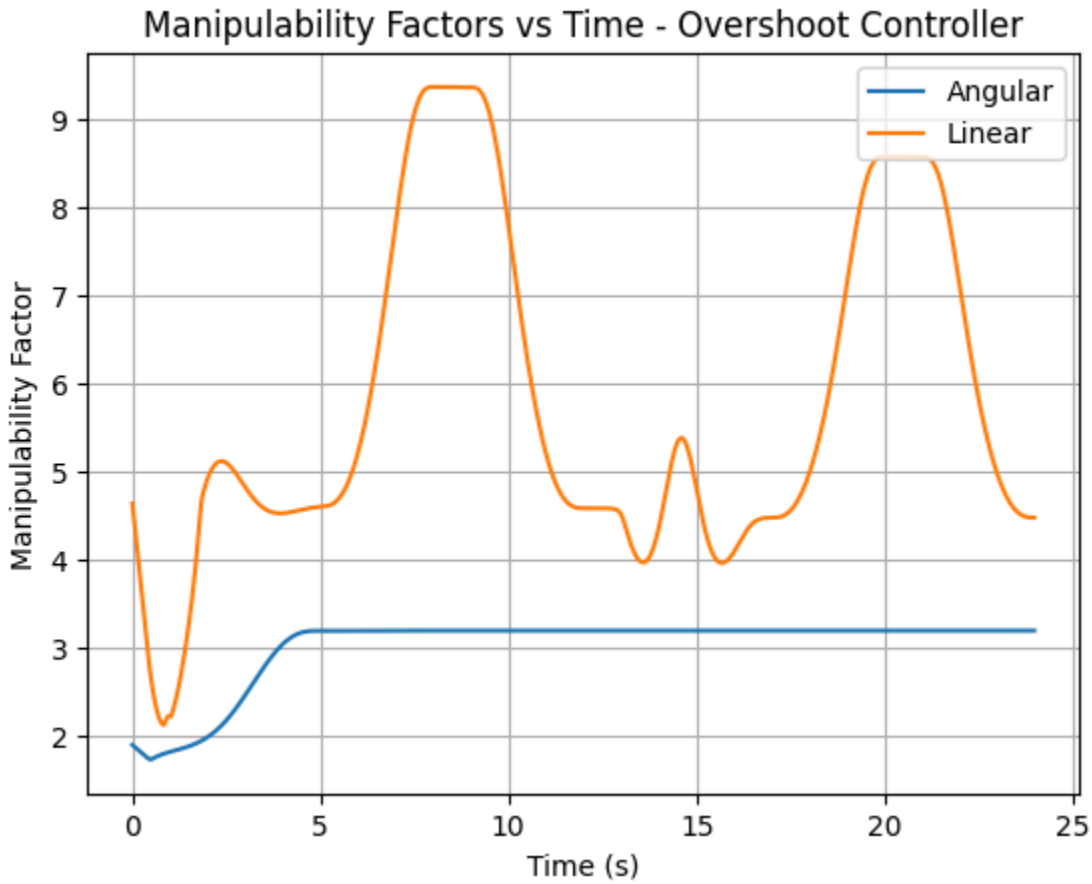
For the overshoot case, the controller used was feedforward + PI, with $K_p = 2.0$ and $K_i = 6.0$.

The initial state of the robot was 45° off the original orientation, $-0.5\,m$ in the x direction, and $+0.3\,m$ in the y direction.

Error plots for the overshoot case are shown below-



Manipulability Factors Plots:



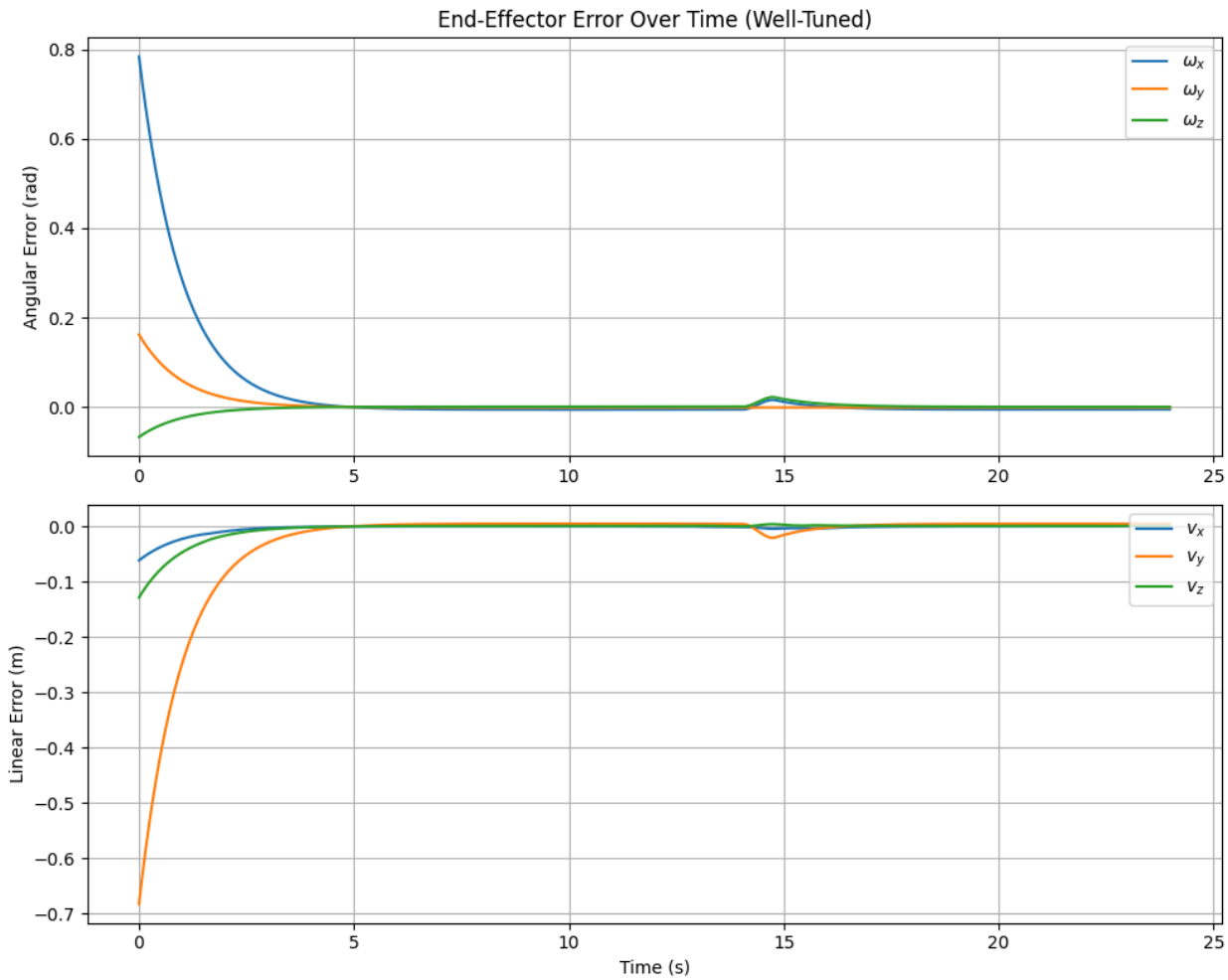
Youtube Video of Overshoot Case:
https://youtu.be/rxJjsLgMGFE

New Task:

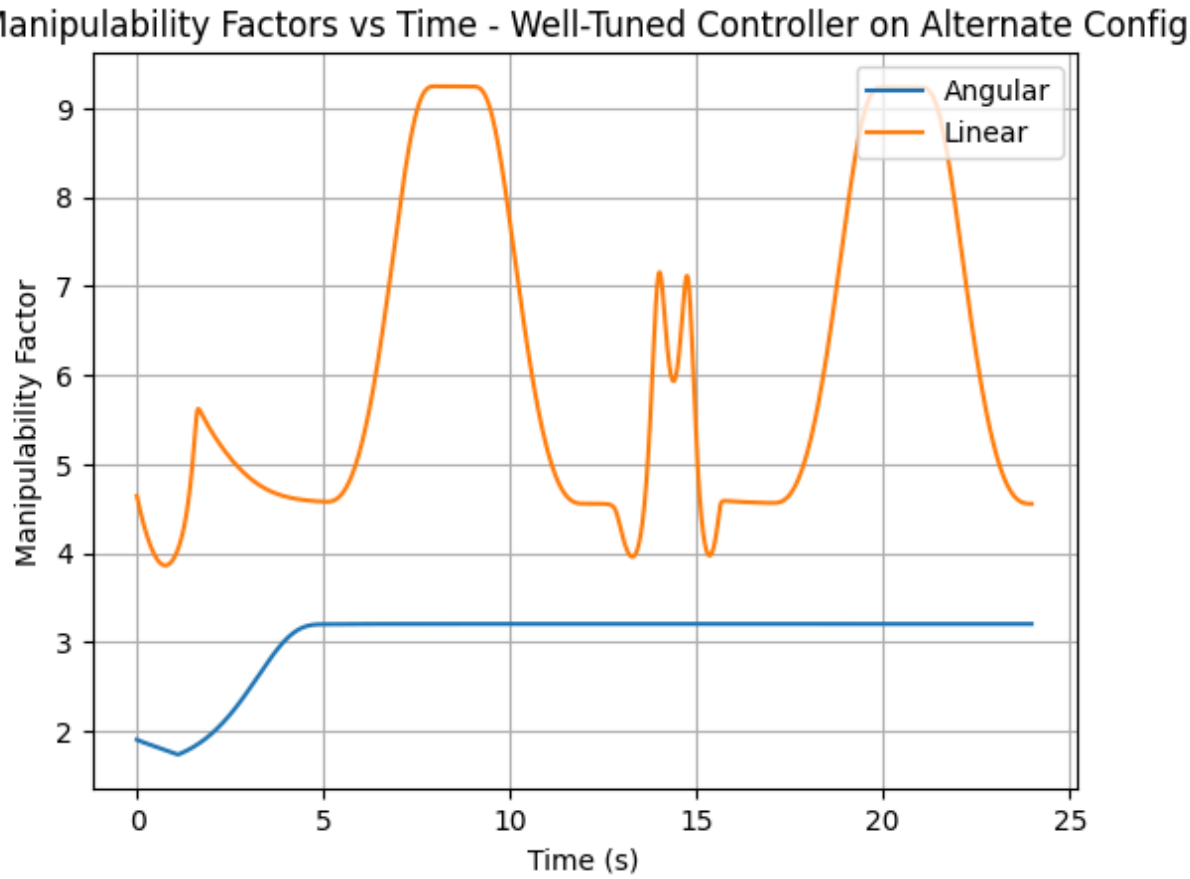
For the new configuration, we had the robot move a block from the position $(1m, 1m, 0)$ to $(-\ 0.5, -\ 1, 0)$, and rotate it by 90° . The same starting configuration for the robot arm was used as for the first test (45° off the target orientation, $-\ 0.5\ m$ in the x direction, and $+\ 0.3\ m$ in the y direction).

Well-Tuned Case:

Error Plots:



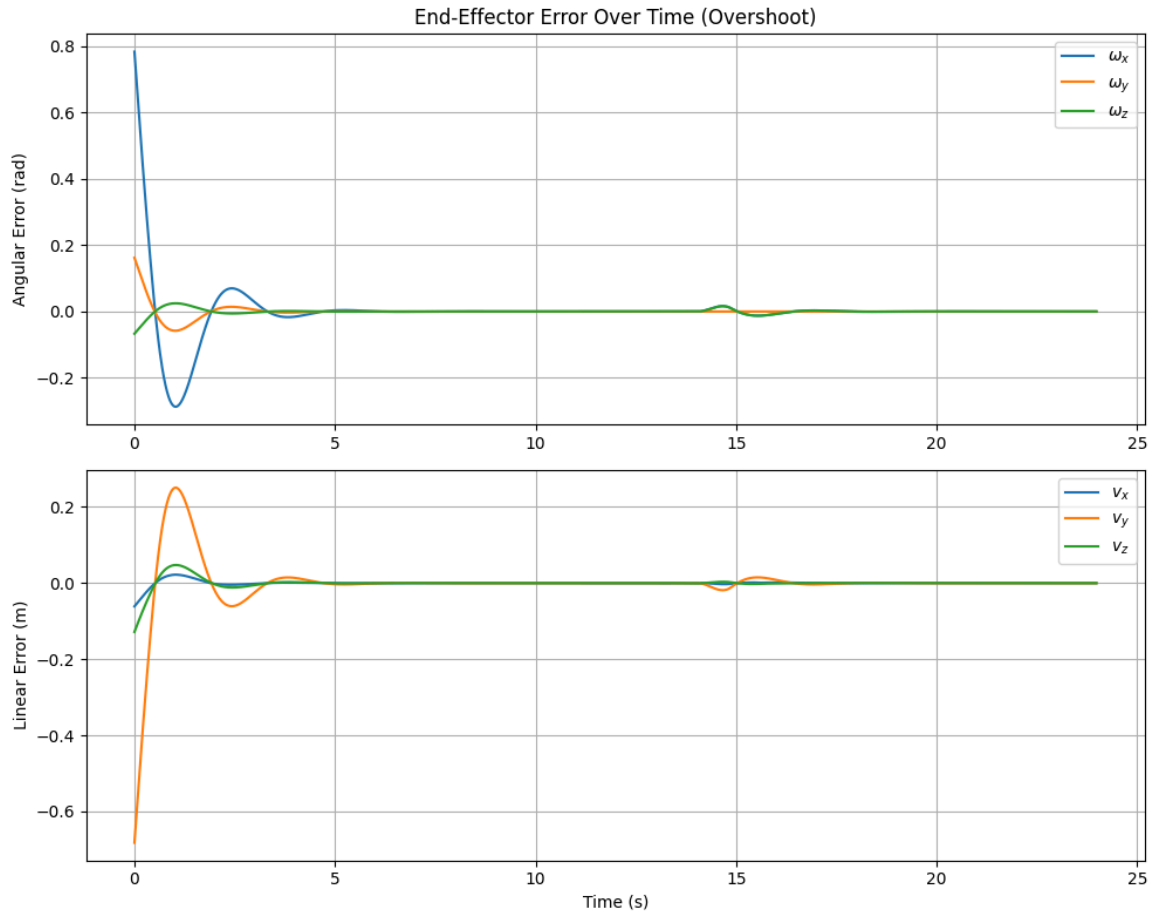
Manipulability Plots:



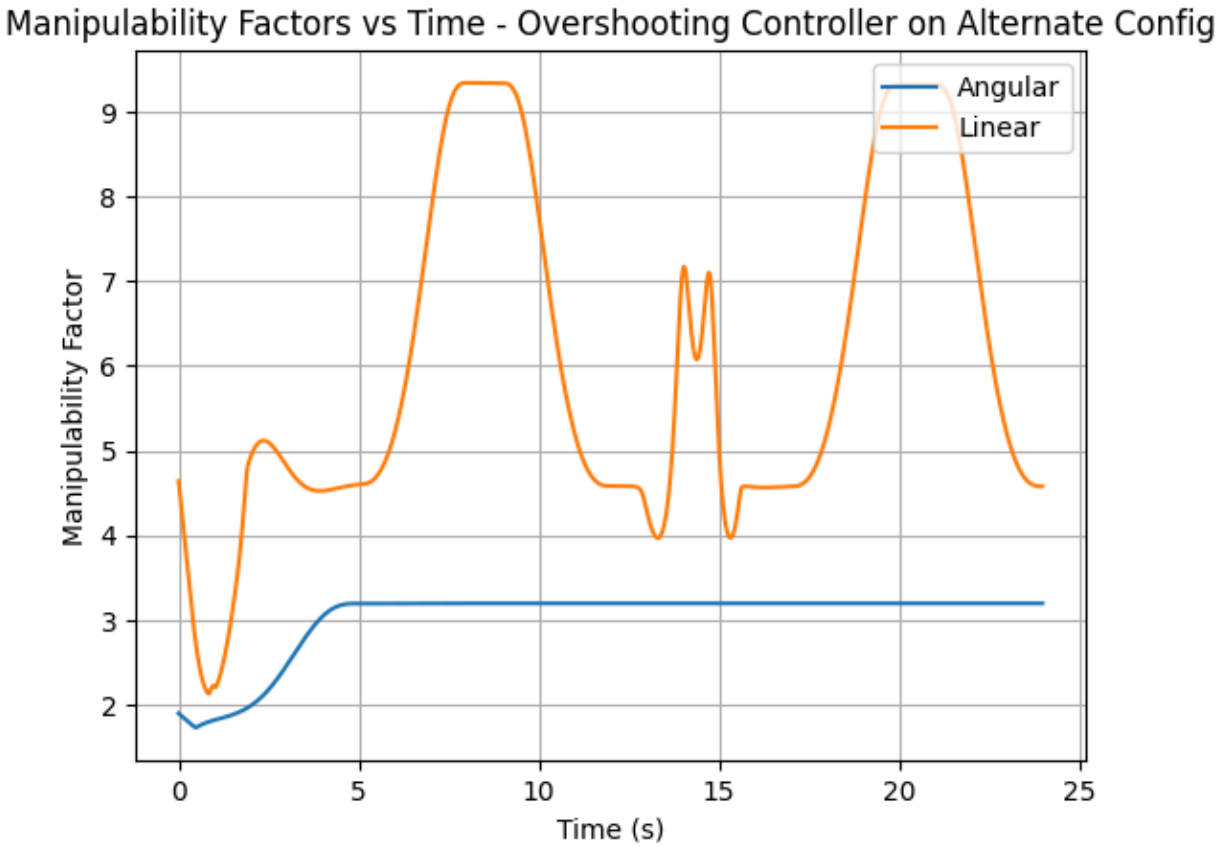
Youtube Video:
https://www.youtube.com/watch?v=_LAYB8AEKD0

Overshoot Case (on new scenario):

Error Plots:



Manipulability Plots:



Youtube Video:
https://youtu.be/l_GbG-t8ROs

Discussion:

1. Adding a larger integrator term causes overshoot, but is more effective at reducing the residual error. In the well-tuned case, there is roughly $2\ mm$ of error (most severe in the z direction), but the case with overshoot has practically no error. A larger integrator term is effective at driving down residual error to zero, but ideally should be matched with an even larger proportional term to quickly bring the system closer to the setpoint. Otherwise, the integrator will "over-wind" before it reaches the setpoint. In our well-tuned case, the proportional constant was $K_p = 1.0$, but the integrator constant was only $K_i = 0.008$. However, in the overshoot case, the integrator was actually larger than the proportional term, at $K_p = 2.0$ and $K_i = 6.0$.
2. The manipulability plots for angular rotation stayed relatively constant throughout the entire process. However, the manipulability graph for end-effector position changed rapidly. It was highest when the arm was mostly extended, since a tiny change in the lower angles (thetas) would substantially impact the end-effector position.
3. Whenever the system starts to approach singularities, the manipulability ellipsoid would change very rapidly as the degrees of freedom were eliminated. It would briefly spike to extremely high levels at a true singularity, and then drop down when the robot got out of the singularity. Sometimes, this exceeds the joint max speeds, causing error to build up in the middle of the path. However, once the arm is back to a more forgiving configuration, the controller could rectify the error. This is most visible in the alternate configuration path. The regular path did not get close enough to a singularity to have error build up in the middle.
4. Velocity control is realistic for moving the robot from location to location. However, velocity control alone would not be enough for a task with strict force limits, such as moving a very fragile cube. Another task that would require sensitive force limits is taking biopsies from a lung cancer patient with a robot arm.
5. Force control would require the knowledge of the mass/inertia of the robot in order to compensate for its own movement in addition to using the Jacobian to calculate the joint torques for a required end-effector torque. The Modern Robotics library has functions like MassMatrix and InverseDynamics to help with modeling the physical robot's inertia. The function EndEffectorForces would also be extremely helpful to calculate the joint torques to produce a desired end effector wrench.
6. Google Gemini was used in this project to debug issues with feedforward only motion (more specifically recognising that the FeedBackControl function did not take in the current actual end effector position and the need to manually determine it based on the initial states vector by running forward kinematics and various matrix multiplications), assist with some programming syntax (extracting information from each matrix and appending or extending into a .csv file), and generate neat error plots for the necessary cases.

Code Release:
The code is available to view at the following github repository:
https://github.com/arobotdinosaur/MAE_204_Final_Project/tree/main

The main notebook which contains all functions (NextState, FeedbackControl,Trajectory Generator along with the plotting and the wrapper script to call the functions) is called "MAE204FinalCode.ipynb"